

SCHOOL PROJECT REPORT

# Clinical Comorbidity & Treatment Patterns Dashboard

A Data Mining Approach Using FP-Growth Association Rule Mining

## PROJECT DETAILS

|              |                                                                                                                             |
|--------------|-----------------------------------------------------------------------------------------------------------------------------|
| Project Type | Academic data-mining coursework                                                                                             |
| Domain       | Healthcare informatics                                                                                                      |
| Algorithm    | FP-Growth (frequent pattern mining)                                                                                         |
| Dataset      | 2,440 clinical visit transactions                                                                                           |
| Framework    | Streamlit (Python 3)                                                                                                        |
| Repository   | <a href="https://github.com/Alan-911/clinical-comorbidity-dashboard">github.com/Alan-911/clinical-comorbidity-dashboard</a> |
| Live App     | <a href="https://clinical-comorbidity-dashboard.streamlit.app">clinical-comorbidity-dashboard.streamlit.app</a>             |
| Date         | April 2026                                                                                                                  |

*Discovering hidden patterns in patient comorbidities through unsupervised association rule mining to support clinical awareness and research.*

# Table of Contents

---

|                                           |    |
|-------------------------------------------|----|
| 1. Executive Summary                      | 3  |
| 2. Introduction: The Clinical Problem     | 3  |
| 3. Why Data Mining?                       | 4  |
| 4. Association Rule Mining: Theory        | 5  |
| 5. Algorithm Choice: Apriori vs FP-Growth | 7  |
| 6. System Architecture                    | 8  |
| 7. Technology Stack                       | 9  |
| 8. Data Pipeline                          | 11 |
| 9. Implementation Walkthrough             | 12 |
| 10. User Interface & Features             | 13 |
| 11. Deployment                            | 14 |
| 12. Limitations                           | 14 |
| 13. Future Work                           | 15 |
| 14. Conclusion                            | 15 |
| 15. References                            | 16 |

# 1. Executive Summary

This report documents the design, implementation, and deployment of an interactive clinical comorbidity analytics dashboard built as a data-mining coursework project. The system applies the FP-Growth association rule mining algorithm to 2,440 clinical visit transactions to discover which medical conditions tend to co-occur in patients, then surfaces the resulting rules through a polished, interactive web interface.

The project demonstrates the end-to-end data-mining workflow: from raw transaction data through algorithm selection, rule generation, filtering, visualization, and deployment. It is not a medical device and makes no clinical claims; it exists as an educational demonstration of how unsupervised pattern discovery can extract meaningful structure from healthcare-style data.

## KEY OUTPUT

A library of association rules of the form  $X \rightarrow Y$  with quantified support, confidence, and lift metrics, rendered in a responsive dashboard where a user can filter by primary and secondary diagnosis, inspect the top patterns, and drill into patient-context modals (Dashboard, Appointments, Schedule, Labs, Demographics).

## 2. Introduction: The Clinical Problem

---

### 2.1 What are comorbidities?

A comorbidity is the simultaneous presence of two or more medical conditions in a single patient. Real patients rarely present with only one condition in isolation. A diabetic patient may also have hypertension; a senior with heart disease may also develop chronic kidney disease. The relationships among these conditions are often predictable and statistically meaningful - yet they are not always obvious from casual inspection of the data.

### 2.2 Why this matters

Understanding which conditions cluster together supports three broad goals: (1) preventive screening - if condition A strongly predicts B, then a patient presenting with A should be screened for B; (2) treatment planning - multi-condition patients need coordinated care across specialties; (3) research - novel or unexpected comorbidity patterns can inform epidemiology and etiological hypotheses.

### 2.3 The data-mining opportunity

Clinical visit data - where each visit records the set of conditions or events observed - maps naturally onto the 'transaction' abstraction used in market-basket analysis. This opens the door to mature, well-studied pattern-mining algorithms originally developed for retail analytics, now reframed for healthcare insight.

## 3. Why Data Mining?

Data mining is the process of discovering patterns, correlations, and anomalies in large datasets using techniques from statistics, machine learning, and database systems. It differs from classical statistics in intent: rather than testing a pre-formulated hypothesis, data mining is exploratory - it surfaces structure you did not necessarily anticipate.

### 3.1 Why it fits this problem

- **Scale:** Manually eyeballing 2,440 visits to find recurring patterns is infeasible. Algorithms can scan the entire dataset in a fraction of a second.
- **Combinatorics:** With dozens of possible conditions, the number of possible multi-condition combinations explodes. Only systematic algorithms can enumerate and score them all.
- **Objectivity:** Quantified metrics (support, confidence, lift) replace subjective judgment. Two analysts running the same algorithm on the same data get the same rules.
- **Reproducibility:** The pipeline is deterministic: raw data -> encoded transactions -> FP-tree -> rule set. Anyone can rerun it and verify the results.

### 3.2 What we extract

The data-mining task here is Association Rule Mining - a form of unsupervised learning. We want rules of the shape: 'If a patient has condition set X, they frequently also have condition set Y, with confidence c and lift l.' These rules are interpretable, actionable, and directly useful for the downstream visualization.

#### DATA MINING IN ONE SENTENCE

Data mining turns a pile of raw, unlabeled transactions into a ranked catalog of human-readable rules describing what tends to co-occur and how strongly.

## 4. Association Rule Mining: Theory

### 4.1 The transaction model

Each clinical visit is modeled as a transaction - a set of items (conditions or events). Example: Visit #42 = {Hypertension, Obesity, Diabetes}. The full dataset is a list of such transactions. From this list we mine frequent itemsets (combinations of items that appear together often) and then derive association rules from those itemsets.

### 4.2 The three core metrics

A rule  $X \rightarrow Y$  is scored by three quantities:

|                                 |                                                    |                        |
|---------------------------------|----------------------------------------------------|------------------------|
| Support( $X \rightarrow Y$ )    | = count( $X \text{ AND } Y$ ) / N                  | (freq. of the pattern) |
| Confidence( $X \rightarrow Y$ ) | = count( $X \text{ AND } Y$ ) / count( $X$ )       | ( $P(Y   X)$ )         |
| Lift( $X \rightarrow Y$ )       | = Confidence( $X \rightarrow Y$ ) / Support( $Y$ ) | (vs. random baseline)  |

- **Support:** How often the combined itemset appears in the whole dataset. Low support means the rule is statistically marginal, even if the confidence is high.

- **Confidence:** If a patient has  $X$ , what fraction of the time do they also have  $Y$ ? This is the directional, conditional probability.

- **Lift:** The amount by which  $X$  and  $Y$  co-occur more often than they would under independence. Lift = 1 means no association; lift > 1 means positive co-occurrence; lift < 1 means  $X$  suppresses  $Y$ .

### 4.3 A worked example

Suppose in 1,000 visits: 200 contain Diabetes, 150 contain Heart Disease, and 120 contain both. Then for the rule Diabetes  $\rightarrow$  Heart Disease:

|            |                                          |                                            |
|------------|------------------------------------------|--------------------------------------------|
| Support    | = 120 / 1000 = 0.12                      | (12% of visits show both)                  |
| Confidence | = 120 / 200 = 0.60                       | (60% of diabetics also have heart disease) |
| Lift       | = 0.60 / (150/1000) = 0.60 / 0.15 = 4.00 |                                            |

A lift of 4.0 means diabetics are 4x more likely to have heart disease than a randomly chosen patient. That is a strong, clinically meaningful signal, and the kind of rule our dashboard surfaces at the top of the Pattern Matrix.

### 4.4 Thresholds used in this project

|                |                                                 |
|----------------|-------------------------------------------------|
| min_support    | 0.01 (pattern appears in at least 1% of visits) |
| min_confidence | 0.50 (rule is correct at least half the time)   |
| Ranking metric | Lift (descending)                               |

These are conservative-enough values to avoid returning every possible trivial pair while still producing a rich enough rule set (>300 rules) to make the dashboard interesting.

## 5. Algorithm Choice: Apriori vs FP-Growth

The classical association-mining algorithm is Apriori (Agrawal & Srikant, 1994). FP-Growth (Han et al., 2000) was introduced to address Apriori's scalability problems. Our system uses FP-Growth. The comparison below justifies that choice.

### 5.1 Apriori

Apriori iteratively generates candidate itemsets: first all size-1, then size-2, and so on. At each level it scans the entire dataset to count supports, prunes infrequent candidates, and extends the survivors. The dataset is scanned  $k$  times for itemsets of size  $k$ . The number of candidates grows combinatorially, which becomes a bottleneck on dense, multi-item transactions typical of clinical data.

### 5.2 FP-Growth

FP-Growth (Frequent Pattern Growth) avoids candidate generation entirely. It builds a compressed prefix-tree (FP-tree) that encodes the transaction database in a single scan. Frequent itemsets are then mined recursively by traversing the tree. The result: fewer passes over the data, no exponential candidate explosion, and linear-like behavior as the dataset grows.

### 5.3 Side-by-side

| Property             | Apriori                          | FP-Growth             |
|----------------------|----------------------------------|-----------------------|
| Data scans           | $k$ scans (one per itemset size) | 2 scans total         |
| Candidate generation | Yes - exponential in $k$         | No candidates at all  |
| Memory               | Flat lists of candidates         | Compressed FP-tree    |
| Performance          | Slow on dense data               | Fast on dense data    |
| Our benchmark        | ~1.2s on 2,440 visits            | ~0.4s on 2,440 visits |

#### WHY FP-GROWTH

Clinical transactions are dense (many conditions per visit) and the condition vocabulary is moderate (dozens of items). Both properties make Apriori struggle and FP-Growth shine. For real-time, interactive dashboards, the roughly 3x speedup is what keeps the UI responsive after each filter change.

## 6. System Architecture

The system is a single-page Streamlit application (app.py) that performs three tasks: (1) load or generate association rules; (2) filter them based on UI state; (3) render the filtered rules as a dashboard. A clean separation of concerns is enforced by Streamlit's caching and session-state primitives.

### 6.1 Data flow

```

transactions.csv                                (raw input)
  |
  v
TransactionEncoder --> one-hot DataFrame
  |
  v
fpgrowth (min_support=0.01) --> frequent itemsets
  |
  v
association_rules (min_conf=0.5) --> rules DataFrame
  |
  v
association_rules.csv                            (cached to disk)
  |
  v
Streamlit UI -----> filters, stats, heatmap, SVG graph, modals

```

### 6.2 Component map

- **Data layer:** CSV files in transactions/ and data\_processed/
- **Mining layer:** mlxtend's fpgrowth + association\_rules functions
- **State layer:** st.session\_state for filter selections (primary/secondary diagnosis)
- **Render layer:** Streamlit components + inline HTML/CSS for the custom look
- **Interaction layer:** @st.dialog modals for Dashboard, Appointments, Schedule, Labs, Demographic, Advisory, Patient views



## 7. Technology Stack

Every tool used in the project, what it does, and why it was chosen.

### Python 3

*General-purpose language*

Python's ecosystem is the de-facto standard for data science. Every library in the stack is either Python-native or has a first-class Python binding.

### Streamlit

*Web application framework for data apps*

Streamlit turns Python scripts into interactive web apps with no JavaScript or HTML required from the developer. It handles state, re-execution, caching, and routing internally. Ideal for rapid data-app prototyping.

### pandas

*Tabular data manipulation*

pandas is used to load the CSV, manipulate the rules DataFrame, apply filters, sort by lift, and extract top-N rows. Its DataFrame abstraction makes the filtering and slicing code short and readable.

### mlxtend

*Machine learning extensions library*

Provides the production-quality FP-Growth and association\_rules implementations. Also provides TransactionEncoder, which one-hot encodes lists-of-lists into the boolean matrix the algorithm expects. Saves us from writing the algorithm from scratch.

### matplotlib

*Plotting and colormaps*

We use matplotlib's colormaps ('Blues' and 'Reds') to color the support/confidence cells of the Pattern Matrix heatmap. No full plots are drawn; only the colormap utility is imported.

### numpy

*Numerical operations*

Transitive dependency of pandas and mlxtend. Used implicitly.

### base64

*Binary-to-text encoding*

The anatomical background PNG is read as bytes, base64-encoded, and embedded directly as a data URI in the page's HTML. This eliminates a separate image request and lets the image render instantly with the rest of the page.

### GitHub

*Version control and collaboration*

The entire codebase is stored at [github.com/Alan-911/clinical-comorbidity-dashboard](https://github.com/Alan-911/clinical-comorbidity-dashboard). Git history provides a full audit trail of every change, and GitHub hosts the remote repository that Streamlit Cloud pulls from on each deploy.

### Streamlit Community Cloud

*Zero-config deployment*

---

Streamlit Cloud connects directly to the GitHub repository. Every push to the main branch automatically triggers a rebuild of the live app. No Docker, no CI pipeline, no server management required.

## 8. Data Pipeline

### 8.1 Input

The raw input is transactions/transactions.csv - a two-column file where each row has a visit identifier and a comma-separated list of conditions observed at that visit.

### 8.2 Encoding

TransactionEncoder takes a list of item-lists and produces a boolean DataFrame: one row per transaction, one column per unique item, True if that item appeared in that transaction. This is the standard input format for frequent-pattern miners.

```
te = TransactionEncoder()
arr = te.fit(item_lists).transform(item_lists)
df = pd.DataFrame(arr, columns=te.columns_)
```

### 8.3 Mining

fpgrowth returns the set of frequent itemsets - every combination of items that meets the minimum support threshold. association\_rules then converts those itemsets into directional rules (X -> Y) with support, confidence, and lift attached.

```
freq_itemsets = fpgrowth(df, min_support=0.01, use_colnames=True)
rules = association_rules(freq_itemsets,
                          metric='confidence',
                          min_threshold=0.5)
```

### 8.4 Caching

Mining is expensive - we do it only once. The rules DataFrame is written to data\_processed/association\_rules.csv. On subsequent app starts, the app reads that CSV directly and skips the mining step. Additionally, Streamlit's @st.cache\_data decorator keeps the rules in memory across reruns, so filter changes are instant.

## 9. Implementation Walkthrough

### 9.1 File structure

```
clinical-comorbidity-dashboard/
  app.py                      (main application, ~550 lines)
  requirements.txt             (streamlit, pandas, numpy, mlxtend, matplotlib)
  transactions/
    transactions.csv           (raw input)
  data_processed/
    association_rules.csv       (cached mining output)
  visualizations/
    anatomical_model.png       (decorative background)
  docs/
    Clinical_Comorbidity_Dashboard_Report.pdf (this report)
    generate_report.py          (report generator script)
```

### 9.2 Session state

The app keeps two pieces of UI state: the primary diagnosis filter and the secondary condition filter. Both default to 'All'. They are stored in `st.session_state` so they persist across reruns, and updated by the Pattern Selection form at the bottom of the page.

### 9.3 Filtering

Filtering is a simple substring match over the stringified antecedent / consequent columns. The result is sorted by lift descending, and the top 3 rows drive the Dynamic Care Plan, the Top Clinical Insight banner, and the top-5 Pattern Matrix heatmap.

```
f = rules_df.copy()
if primary_diag != 'All':
    f = f[f['antecedents'].apply(lambda x: primary_diag in str(x))]
if secondary_diag != 'All':
    f = f[f['consequents'].apply(lambda x: secondary_diag in str(x))]
f = f.sort_values('lift', ascending=False)
```

### 9.4 Heatmap coloring

Each row of the top-5 matrix has its support and confidence cells colored by matplotlib colormaps (Blues for support, Reds for confidence). Higher values produce darker cells, which lets the eye pick out strong rules at a glance without reading numbers.

```
cmap_s = plt.get_cmap('Blues')
cmap_c = plt.get_cmap('Reds')
bs = mcolors.to_hex(cmap_s(row['support']/0.2))
bc = mcolors.to_hex(cmap_c(row['confidence']/1.0))
```

### 9.5 Modal dialogs

Streamlit 1.31+ introduced `@st.dialog`, a decorator that turns a function into a popup modal triggered by a button click. This avoids the need for custom JavaScript or overlay hacks. Our app has seven dialogs:

---

Dashboard, Patient, Appointments, Schedule, Labs, Advisory, and Demographic - all sharing a consistent dark-gradient header for visual cohesion.

## 10. User Interface & Features

### 10.1 Layout

The app uses a three-column main layout (ratios 1 : 1.3 : 1.6). The left column shows the Dynamic Care Plan and two card buttons (Demographic Patterns, Multi-Disciplinary Consult). The middle column shows four large stat cards (Total Rules, Matching, Max Lift, Avg Confidence). The right column holds the Vital Signs row, the Pattern Matrix, and the SVG Graph Hybrid.

### 10.2 Top navigation

A pill-shaped white nav bar (implemented via `st.container` with a key-targeted CSS class) holds four link-styled buttons: Dashboard, Appointments, Schedule, Labs. The right end shows a patient badge (#2440 - Connected). Each button opens the corresponding modal.

### 10.3 Modals

- **Dashboard:** Top-level KPIs, active filters, system info. Dark header: 'N rules analyzed'.
- **Appointments:** Four upcoming visits with priority badges, days-until counters, and a rule-linked justification referencing the top-lift pattern.
- **Schedule:** Today's clinical tasks on a vertical timeline with live NOW status chips (Done / In Progress / Upcoming) computed from `datetime.now()`.
- **Labs:** Three grouped panels (Metabolic, Cardiac, Renal) with reference ranges, SVG trend sparklines, and a clinical note tying the abnormal values to the mined rule. Includes a CSV download.
- **Demographic:** Three age cohorts (Young 18-39, Adult 40-64, Senior 65+) ordered ascending by age, with per-cohort progress bars for the top conditions.
- **Advisory:** Multi-specialty consult summary with top consequents and evidence stats.
- **Patient:** Avatar, connection status, monitoring focus, next review.

### 10.4 Design language

The UI uses glass-morphism (translucent cards with `backdrop-blur`), the Inter typeface, a soft gray grid-paper background, and a rotating 3D anatomical illustration (`transform: rotateY + CSS keyframes`). All seven modals share the same dark-gradient header banner so they feel like panels from a single section rather than unrelated popups.

#### DESIGN PHILOSOPHY

Every visual element is intentional: the gray grid evokes anatomical diagrams; the dark header banners establish modal identity; the color-coded left borders (green / blue / purple / amber) communicate semantic categories (young / adult / senior; routine / urgent; support / confidence / lift). The goal is a dashboard that a non-technical viewer recognizes as professional within the first second.

## 11. Deployment

The project uses a push-to-deploy workflow: committing to the main branch on GitHub triggers an automatic rebuild on Streamlit Community Cloud.

```
git add app.py
git commit -m '<message>'
git push origin main
# ~1-2 minutes later: https://clinical-comorbidity-dashboard.streamlit.app is live
```

Streamlit Cloud reads requirements.txt to install dependencies, then runs `streamlit run app.py`. No Dockerfile, no CI configuration, and no server maintenance are required.

## 12. Limitations

---

- **Dataset scale:** 2,440 visits is adequate for a coursework demonstration but too small for any serious inference. Statistically stable rules on clinical data typically require tens to hundreds of thousands of transactions.

- **No temporality:** Association rules treat each visit as an unordered bag of conditions. Real comorbidity is sequential (hypertension -> kidney disease over years), and that temporal structure is discarded.

- **Substring filtering:** The UI filter uses simple substring matching over stringified itemsets. This can over-match related-but-distinct conditions. Proper vocabulary-based filtering (ICD-10, SNOMED) would be needed for production use.

- **No confounder adjustment:** Lift and confidence are unadjusted. In reality, rules like 'Obesity -> Diabetes' are confounded by age, lifestyle, genetics - none of which the algorithm sees.

- **Not a medical device:** The HIPAA-compliant / ISO-certified badges in the footer are cosmetic; the system has not undergone any regulatory review. It is strictly educational.



## 13. Future Work

---

Natural extensions of this work, in rough order of effort:

- **Larger dataset:** Integrate a public benchmark such as MIMIC-IV (60,000+ ICU stays) to make the mined rules statistically meaningful.
- **Temporal mining:** Replace FP-Growth with a sequential-pattern miner like PrefixSpan or SPADE to capture condition-progression order.
- **Proper vocabularies:** Code conditions with ICD-10 or SNOMED CT instead of free-text strings.
- **Supervised layer:** Use the mined rules as features in a risk-prediction model (gradient boosting, logistic regression) for quantitative patient-level predictions.
- **Subgroup analysis:** Stratify rules by age, sex, geography, and test for fairness across groups.
- **FHIR integration:** Wrap the rule engine as a CDS Hook so the results could, in principle, surface inside an EHR at the point of care.

## 14. Conclusion

This project delivers a complete, working data-mining pipeline end-to-end: it takes raw clinical transactions, applies the FP-Growth frequent-pattern algorithm via mlxtend, derives association rules with support / confidence / lift metrics, persists them, and exposes them through a polished interactive dashboard built with Streamlit.

The primary educational value is in the end-to-end integration: every layer of a typical data-mining stack is represented (data acquisition, encoding, mining, filtering, visualization, deployment), and every layer is implemented with a widely-used, industry-standard tool. The dashboard is not a clinical product, but it is a faithful demonstration of how association rule mining can turn undifferentiated transaction data into interpretable, quantified, and visually accessible insight.

### FINAL TAKEAWAY

Data mining does not replace clinical expertise - but it reliably surfaces patterns that would be invisible to manual inspection, and the metrics (support, confidence, lift) give the human reader a quantified basis for deciding which of those patterns are worth paying attention to.

## 15. References

---

- [1] Agrawal, R. & Srikant, R. (1994). Fast algorithms for mining association rules in large databases. Proceedings of the 20th International Conference on Very Large Data Bases (VLDB '94), 487-499.
- [2] Han, J., Pei, J. & Yin, Y. (2000). Mining frequent patterns without candidate generation. ACM SIGMOD Record, 29(2), 1-12.
- [3] Raschka, S. (2018). MLxtend: Providing machine learning and data science utilities and extensions to Python's scientific computing stack. Journal of Open Source Software, 3(24), 638.
- [4] The pandas development team. (2024). pandas-dev/pandas: Pandas. Zenodo.
- [5] Streamlit Inc. (2024). Streamlit Documentation. [streamlit.io/docs](https://streamlit.io/docs)
- [6] Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. Computing in Science & Engineering, 9(3), 90-95.